/router

Manage the model-router cluster (/router on|off|status|test|weights|boundaries|tiers|reload|help)

Overview

`/router` controls the Model Router provider, which classifies each prompt across multiple scoring dimensions and routes it to a tier of concrete providers. The command can turn routing on or off for the session, show the current tier and configuration, classify a test prompt, and list the weights, boundaries, and tiers currently loaded. Some advertised tuning subcommands are not implemented in this build; see the limitations note below.

Syntax

```
/router               Show the help table (same as /router help)
/router on            Enable routing for this session
/router off           Disable routing for this session
/router status        Show routing state and current tier
/router test <prompt>  Classify a prompt and print the tier decision
/router weights       Show the routing weight table
/router boundaries    Show the tier boundary thresholds
/router tiers         Show the configured tier membership
/router reload        Re-read router settings from ragent.json
/router help          Show the help table
```

Options / Subcommands

Form	Description
`/router`	Alias of `/router help`; prints the help table
`/router on`	Sets the in-session routing flag; routing takes effect on the next prompt
`/router off`	Clears the in-session routing flag; subsequent prompts run on the active model
`/router status`	Shows whether routing is enabled and the current selected tier
`/router test <prompt>`	Classifies the prompt over 15 scoring dimensions and prints the composite score, chosen tier, and requires_vision flag; also updates the in-session current tier
`/router weights`	Prints the per-dimension weights used by the classifier
`/router boundaries`	Prints the tier boundary thresholds used to map composite scores to tiers
`/router tiers`	Prints the tier membership lists (fast/standard/deep and so on)
`/router reload`	Re-reads `.ragent/ragent.json` (falling back to the global config directory), parses `provider.router` into the router configuration, and resets the current tier
`/router help`	Prints the help table

Any other subcommand prints the unknown-subcommand warning.

Examples

`/router on`

Enables model routing for this session.

`/router test refactor the config parser to reduce allocations`

Prints a 15-dimension score table followed by the composite score, the selected tier, and whether the prompt requires a vision-capable model.

`/router tiers`

Lists which models belong to each tier under the loaded configuration.

`/router reload`

Re-reads `.ragent/ragent.json` after you edited the `provider.router` section, so the new weights and boundaries take effect without restarting the TUI.

`/router stats`

Not a supported form in this build - prints the unknown-subcommand warning.

Output

- `/router status`: a status block with the on/off state and the tier currently selected for the next routing decision.
- `/router test <prompt>`: one line per scoring dimension with its score, then the composite score, the tier decision, and the `requires_vision` flag. The in-session current tier is updated as a side effect.
- `/router weights` and `/router boundaries`: the tables loaded from the last configuration read (defaults if no `provider.router` section is present).
- `/router on` / `off`: a short confirmation message.

Current limitations

- The `/router help` text also advertises `tier <name> set|add|remove`, `weights set|reset`, `boundaries set`, and `stats` (including `stats reset`), but those subcommands are **not implemented** in this build; they fall through to the unknown-subcommand warning.
- Per-dimension weight overrides and boundary thresholds cannot be changed from the command line. Edit `provider.router.weights` and `provider.router.boundaries` in `ragent.json`, then run `/router reload`.
- `/router on` and `/router off` set an in-session flag only; they do not persist to `ragent.json`.

Related

- `/model` - pick models; with the `router` provider the setup dialog seeds tiers
- `/provider` - configure the router's concrete backing providers
- `provider.router.weights / provider.router.boundaries in ragent.json` - the configuration-file surface for weight and boundary tuning